

Comparative Empirical Evaluation of Feedforward Neural Network Training Algorithms: Stochastic Gradient Descent, Scaled Conjugate Gradient, and LeapFrog Optimization

T.D. Heath (Student Number: 24888923)

Department of Mathematical Sciences (Computer Science Division)

Stellenbosch University, Stellenbosch, South Africa

Email: 24888923@sun.ac.za

Abstract—Supervised learning in feedforward neural networks requires solving non-convex parameter optimization problems over complex loss surfaces. First-order gradient descent methods frequently suffer from slow convergence rates, high sensitivity to learning rate selection, and ill-conditioned curvature oscillations. This paper presents a comparative empirical evaluation of three feedforward neural network training algorithms: stochastic gradient descent (SGD) with momentum, scaled conjugate gradient (SCG) optimization, and the physical trajectory LeapFrog optimizer. All neural network architectures, backpropagation gradients, and optimization procedures were implemented from scratch in pure Python. Evaluation was conducted across three benchmark classification problems (Iris, Pima Indians Diabetes, and Glass Identification) and three function approximation problems (1D Sine Wave, 2D Franke function, and Housing price estimation) using five-fold cross-validation. Experimental results demonstrate that SCG achieves superior convergence efficiency, reaching low error thresholds in significantly fewer function evaluations compared to first-order SGD. Non-parametric statistical evaluation using the Friedman omnibus test ($F_F = 8.5849, p = 0.0068$) and Nemenyi post-hoc critical difference analysis confirmed the statistically significant performance advantage of second-order Hessian scaling over standard gradient descent.

I. INTRODUCTION

Supervised training of feedforward neural networks involves adjusting model weight vectors to minimize a scalar loss function defined over training instances. The optimization landscape of deep and shallow neural networks is inherently non-convex, characterized by steep ravines, saddle points, and extensive plateaus [1]. First-order optimization algorithms such as stochastic gradient descent (SGD) rely exclusively on localized gradient vectors. Consequently, SGD performance depends critically on hyperparameter tuning, particularly the learning rate and momentum coefficient; suboptimal selections frequently induce either slow asymptotic convergence or divergent numerical oscillations [2]. To incorporate curvature information without the computational bottleneck of explicit line searches typical of standard conjugate gradient methods, second-order techniques approximate the Hessian matrix. The Scaled Conjugate Gradient (SCG) algorithm, introduced by Möller [1], calculates conjugate search directions

while estimating Hessian-vector products via finite difference approximations, bypassing line searches entirely through a dynamic step-size regulation mechanism. Alternatively, physical dynamic trajectory methods, such as the LeapFrog algorithm developed by Snyman [2], [3], model the parameter vector as a physical particle traversing a conservative potential energy field. LeapFrog employs dynamic velocity updates and kinetic energy monitoring with valley-wall reset strategies, navigating ill-conditioned error surfaces without requiring gradient step-size line searches. This study presents a comparative empirical analysis of SGD with momentum, SCG, and LeapFrog optimization across six supervised benchmark problems of varying dimensionality and complexity. This investigation addresses three primary research questions: first, whether SCG achieves faster convergence and lower generalization error than SGD across varying problem complexities; second, how LeapFrog optimization performs on ill-conditioned error landscapes relative to conventional gradient methods; and third, how hidden-unit capacity influences algorithm-specific convergence stability and generalization performance. The remainder of this paper is organized as follows. Section II details the mathematical foundations of feedforward neural networks, analytical backpropagation, SGD, SCG, LeapFrog, and non-parametric statistical evaluation. Section III describes the modular software architecture implemented natively in Python. Section IV establishes the empirical experimental methodology, dataset specifications, preprocessing pipelines, and hyperparameter search configurations. Section V presents numerical results, convergence profiles, critical difference diagrams, and an analytical discussion addressing each research question. Finally, Section VI concludes the paper and highlights directions for future work.

II. BACKGROUND

This section provides the mathematical formulations governing feedforward neural networks, backpropagation, and the three optimization algorithms evaluated in this study.

A. Feedforward Neural Network Architecture

Consider a single hidden layer feedforward neural network mapping an input vector $\mathbf{x} \in \mathbb{R}^D$ to an output vector $\hat{\mathbf{y}} \in \mathbb{R}^K$. Let n_h denote the number of hidden layer units. The forward transformation is defined by:

$$\mathbf{a}_1 = \mathbf{W}^{(1)}\mathbf{x} + \mathbf{b}^{(1)}, \quad \mathbf{y}_1 = f_h(\mathbf{a}_1) \quad (1)$$

$$\mathbf{a}_2 = \mathbf{W}^{(2)}\mathbf{y}_1 + \mathbf{b}^{(2)}, \quad \hat{\mathbf{y}} = f_o(\mathbf{a}_2) \quad (2)$$

where $\mathbf{W}^{(1)} \in \mathbb{R}^{n_h \times D}$ and $\mathbf{b}^{(1)} \in \mathbb{R}^{n_h}$ represent hidden layer weights and biases, while $\mathbf{W}^{(2)} \in \mathbb{R}^{K \times n_h}$ and $\mathbf{b}^{(2)} \in \mathbb{R}^K$ represent output layer weights and biases. The hidden activation function $f_h(a)$ is chosen as the logistic sigmoid function:

$$f_h(a) = \frac{1}{1 + e^{-a}} \quad (3)$$

The sigmoid activation satisfies the universal approximation property for single-hidden-layer networks [5] and provides C^∞ smoothness, guaranteeing continuous gradients and well-defined curvature required by second-order and dynamic trajectory optimizers.

For function approximation (regression) tasks, the output activation function is the linear identity $f_o(a) = a$. Paired with mean squared error (MSE), this corresponds to the maximum likelihood objective under an assumption of homoscedastic Gaussian noise, regularized by an L_2 weight decay term with hyperparameter $\gamma > 0$:

$$E(\mathbf{w}) = \frac{1}{2N} \sum_{i=1}^N \|\mathbf{y}_i - \hat{\mathbf{y}}_i\|^2 + \frac{\gamma}{2} \|\mathbf{w}\|^2 \quad (4)$$

For multi-class classification tasks with C classes, the output activation is the normalized softmax function:

$$f_o(\mathbf{a})_c = \frac{e^{a_c}}{\sum_{k=1}^C e^{a_k}}, \quad c \in \{1, \dots, C\} \quad (5)$$

paired with the categorical cross-entropy loss function:

$$E(\mathbf{w}) = -\frac{1}{N} \sum_{i=1}^N \sum_{c=1}^C y_{i,c} \ln(\hat{y}_{i,c}) + \frac{\gamma}{2} \|\mathbf{w}\|^2 \quad (6)$$

where $\mathbf{w} \in \mathbb{R}^M$ represents the flattened parameter vector of total dimension $M = n_h(D + 1) + K(n_h + 1)$.

Both configurations establish canonical link pairings (linear with MSE, softmax with cross-entropy). This formulation prevents output-layer gradient saturation and simplifies the analytic parameter gradients $\nabla E(\mathbf{w}) = \mathbf{g}(\mathbf{w})$ computed via backpropagation, yielding an identical linear output error vector δ_2 for both objectives:

$$\delta_2 = \frac{1}{N} (\hat{\mathbf{y}} - \mathbf{y}) \quad (7)$$

The hidden layer error vector δ_1 is obtained via backward propagation through the transpose weight matrix $\mathbf{W}^{(2)}$:

$$\delta_1 = (\mathbf{W}^{(2)T} \delta_2) \odot f'_h(\mathbf{a}_1) \quad (8)$$

where $\odot$ represents the element-wise Hadamard product, and $f'_h(a) = y_1(1 - y_1)$. Parameter gradients are then evaluated as:

$$\frac{\partial E}{\partial \mathbf{W}^{(2)}} = \delta_2 \mathbf{y}_1^T + \gamma \mathbf{W}^{(2)}, \quad \frac{\partial E}{\partial \mathbf{b}^{(2)}} = \delta_2 \quad (9)$$

$$\frac{\partial E}{\partial \mathbf{W}^{(1)}} = \delta_1 \mathbf{x}^T + \gamma \mathbf{W}^{(1)}, \quad \frac{\partial E}{\partial \mathbf{b}^{(1)}} = \delta_1 \quad (10)$$

B. Stochastic Gradient Descent with Momentum

Stochastic gradient descent updates the parameter vector $\mathbf{w}_t$ along the negative gradient direction. Incorporating a momentum coefficient $\alpha \in [0, 1)$ stabilizes weight updates across successive iterations:

$$\mathbf{v}_{t+1} = \alpha \mathbf{v}_t - \eta \nabla E(\mathbf{w}_t) \quad (11)$$

$$\mathbf{w}_{t+1} = \mathbf{w}_t + \mathbf{v}_{t+1} \quad (12)$$

where $\eta > 0$ represents the step-size learning rate, and $\mathbf{v}_t$ denotes the accumulation velocity vector.

C. Scaled Conjugate Gradient Optimization

The scaled conjugate gradient algorithm, proposed by Møller [1], combines conjugate direction search with second-order Hessian scaling. Let $\mathbf{p}_k$ denote the search direction at iteration k , and $\mathbf{r}_k = -\mathbf{g}_k = -\nabla E(\mathbf{w}_k)$ represent the residual vector. Second-order information is estimated via a finite-difference approximation along direction $\mathbf{p}_k$:

$$\mathbf{s}_k = \frac{\nabla E(\mathbf{w}_k + \sigma_k \mathbf{p}_k) - \nabla E(\mathbf{w}_k)}{\sigma_k} \quad (13)$$

where $\sigma_k = \sigma / \|\mathbf{p}_k\|$ with scalar perturbation $\sigma = 10^{-4}$. To ensure positive definiteness of the Hessian approximation, SCG scales the curvature scalar $\delta_k = \mathbf{p}_k^T \mathbf{s}_k$ using scale parameter $\lambda_k > 0$:

$$\bar{\delta}_k = \delta_k + (\lambda_k - \bar{\lambda}_k) \|\mathbf{p}_k\|^2 \quad (14)$$

If $\bar{\delta}_k \leq 0$, the scaling factor is updated according to:

$$\bar{\lambda}_k = 2 \left(\lambda_k - \frac{\bar{\delta}_k}{\|\mathbf{p}_k\|^2} \right), \quad \lambda_k = \bar{\lambda}_k \quad (15)$$

The optimal step size α_k along direction $\mathbf{p}_k$ is evaluated directly:

$$\alpha_k = \frac{\mathbf{p}_k^T \mathbf{r}_k}{\bar{\delta}_k} \quad (16)$$

A comparison parameter Δ_k measures the ratio of actual loss reduction to quadratic approximation model reduction:

$$\Delta_k = \frac{2\bar{\delta}_k [E(\mathbf{w}_k) - E(\mathbf{w}_k + \alpha_k \mathbf{p}_k)]}{(\mathbf{p}_k^T \mathbf{r}_k)^2} \quad (17)$$

If $\Delta_k \geq 0$, the step is accepted ($\mathbf{w}_{k+1} = \mathbf{w}_k + \alpha_k \mathbf{p}_k$), and the scale parameter λ_k is reduced. If $\Delta_k < 0.25$, λ_k is increased. Conjugate direction updates follow the Polak-Ribière formula:

$$\beta_k = \frac{\|\mathbf{r}_{k+1}\|^2 - \mathbf{r}_{k+1}^T \mathbf{r}_k}{\mathbf{p}_k^T \mathbf{r}_k}, \quad \mathbf{p}_{k+1} = \mathbf{r}_{k+1} + \beta_k \mathbf{p}_k \quad (18)$$

D. LeapFrog Optimization

The LeapFrog optimization algorithm, introduced by Snyman [2], [3], conceptualizes function minimization as tracking a unit-mass particle moving in a conservative potential force field $\mathbf{a}(\mathbf{w}) = -\nabla E(\mathbf{w})$. The continuous physical state equations are integrated using a half-step velocity formulation:

$$\mathbf{v}_{t+1/2} = \mathbf{v}_{t-1/2} + \mathbf{a}(\mathbf{w}_t)\Delta t \quad (19)$$

$$\mathbf{w}_{t+1} = \mathbf{w}_t + \mathbf{v}_{t+1/2}\Delta t \quad (20)$$

where Δt represents the integration timestep. Full-step velocity is estimated as:

$$\mathbf{v}_t = \frac{1}{2}(\mathbf{v}_{t-1/2} + \mathbf{v}_{t+1/2}) \quad (21)$$

To prevent particle overshooting when traversing steep valley walls, LeapFrog applies an interfering strategy. If the velocity vector forms an obtuse angle with the acceleration vector ($\mathbf{v}_t^T \mathbf{a}_t < 0$) or if loss increases ($E(\mathbf{w}_{t+1}) > E(\mathbf{w}_t)$), the particle has passed a local potential minimum. The algorithm resets velocity to zero at the midpoint position, halves the integration step size ($\Delta t \leftarrow 0.5\Delta t$), and restarts acceleration from the valley floor.

E. Non-Parametric Statistical Evaluation Framework

To compare multiple optimization algorithms across diverse benchmark datasets without imposing normality assumptions, this study follows the statistical evaluation protocol described by Demšar [4]. For N datasets and K algorithms, performance metrics are converted to ranks per dataset, where rank 1 corresponds to the best performance. Let r_i^j denote the rank of algorithm j on dataset i . The average rank R_j is calculated as:

$$R_j = \frac{1}{N} \sum_{i=1}^N r_i^j \quad (22)$$

The omnibus Friedman chi-square statistic χ_F^2 is defined by:

$$\chi_F^2 = \frac{12N}{K(K+1)} \left[\sum_{j=1}^K R_j^2 - \frac{K(K+1)^2}{4} \right] \quad (23)$$

Iman and Davenport showed that χ_F^2 exhibits conservative bias and proposed the F -distribution derivative statistic F_F :

$$F_F = \frac{(N-1)\chi_F^2}{N(K-1) - \chi_F^2} \quad (24)$$

which follows an F -distribution with $K-1$ and $(N-1)(K-1)$ degrees of freedom. If the null hypothesis of equal performance is rejected, the Nemenyi post-hoc test determines pairwise statistical differences. The critical difference (CD) threshold at significance level $\alpha = 0.05$ is given by:

$$CD = q_\alpha \sqrt{\frac{K(K+1)}{6N}} \quad (25)$$

where critical value $q_{0.05} = 2.343$ for $K = 3$ algorithms. Performance differences between two algorithms are statistically significant if their average rank difference exceeds CD .

III. IMPLEMENTATION

All neural network structures, matrix calculations, analytical gradient procedures, and optimization algorithms were implemented from scratch in Python using basic NumPy array operations. High-level deep learning wrappers (such as PyTorch or TensorFlow) were excluded to maintain full algorithmic transparency.

The software architecture is modularized into four distinct packages inside `src/`:

1. `neural_network.py`: Implements the `FeedforwardNeuralNetwork` class, handling Glorot parameter initialization, forward propagation, output activation transformations, regularized loss calculation, and analytical backpropagation gradients.
2. `optimizers/`: Contains self-contained optimizer classes (`SGDOptimizer`, `SCGOptimizer`, `LeapFrogOptimizer`).
3. `preprocessing.py` and `metrics.py`: Implement custom min-max scaling, z-score standardization, one-hot target encoding, and evaluation metrics.
4. `statistical_tests.py`: Implements the Friedman omnibus test, F_F transformation, Nemenyi critical difference calculation, and Critical Difference Plot generation.

Mathematical precision was verified by comparing analytical backpropagation gradients against numerical finite-difference gradients evaluated via central differences:

$$g_{\text{num}}(\mathbf{w}_i) = \frac{E(\mathbf{w} + \epsilon \mathbf{e}_i) - E(\mathbf{w} - \epsilon \mathbf{e}_i)}{2\epsilon} \quad (26)$$

Gradient check validation produced relative error norms lower than 10^{-9} across all network dimensions, verifying exact mathematical correctness. Algorithms 1 and 2 summarize the execution steps for Scaled Conjugate Gradient and LeapFrog optimization.

IV. EMPIRICAL PROCESS

This section describes the benchmark datasets, data preprocessing procedures, hidden unit capacity grid searches, hyperparameter tuning spaces, and five-fold cross-validation framework.

A. Benchmark Datasets

Six supervised learning benchmark datasets spanning low, medium, and high complexity were selected. Table I summarizes the domain characteristics, sample counts (N), feature dimensions (D), and target distributions for all six datasets.

The classification benchmarks comprise the Iris flower dataset ($N = 150, D = 4, C = 3$), the Pima Indians Diabetes dataset ($N = 768, D = 8, C = 2$), and the multi-class Glass Identification dataset ($N = 214, D = 9, C = 6$). The regression benchmarks comprise a noisy 1D sine wave function ($y = \sin(2\pi x) + \epsilon, \epsilon \sim \mathcal{N}(0, 0.01)$), Franke's bivariate test function ($N = 400, D = 2$), and a standardized subset of real-world housing price attributes ($N = 506, D = 9$).

Algorithm 1 Scaled Conjugate Gradient (Møller 1993)

```

1: Initialize  $\mathbf{w}_1$ ,  $\sigma = 10^{-4}$ ,  $\lambda_1 = 10^{-6}$ ,  $\bar{\lambda}_1 = 0$ , success = True.
2: Set  $\mathbf{p}_1 = \mathbf{r}_1 = -\nabla E(\mathbf{w}_1)$ ,  $k = 1$ .
3: while  $k \leq k_{\max}$  and  $\|\mathbf{r}_k\| > \text{tol}$  do
4:   if success == True then
5:     Compute  $\mathbf{s}_k = \frac{\nabla E(\mathbf{w}_k + \sigma_k \mathbf{p}_k) - \nabla E(\mathbf{w}_k)}{\sigma_k}$  where  $\sigma_k = \frac{\sigma}{\|\mathbf{p}_k\|}$ .
6:     Compute  $\delta_k = \mathbf{p}_k^T \mathbf{s}_k$ .
7:   end if
8:   Scale Hessian:  $\bar{\delta}_k = \delta_k + (\lambda_k - \bar{\lambda}_k) \|\mathbf{p}_k\|^2$ .
9:   if  $\bar{\delta}_k \leq 0$  then
10:     $\bar{\lambda}_k = 2(\lambda_k - \frac{\bar{\delta}_k}{\|\mathbf{p}_k\|^2})$ ,  $\lambda_k = \bar{\lambda}_k$ .
11:   end if
12:   Compute step size  $\alpha_k = \frac{\mathbf{p}_k^T \mathbf{r}_k}{\bar{\delta}_k}$ .
13:   Compute comparison parameter  $\Delta_k = \frac{2\bar{\delta}_k [E(\mathbf{w}_k) - E(\mathbf{w}_k + \alpha_k \mathbf{p}_k)]}{(\mathbf{p}_k^T \mathbf{r}_k)^2}$ .
14:   if  $\Delta_k \geq 0$  then
15:     Update  $\mathbf{w}_{k+1} = \mathbf{w}_k + \alpha_k \mathbf{p}_k$ ,  $\mathbf{r}_{k+1} = -\nabla E(\mathbf{w}_{k+1})$ .
16:     Set  $\bar{\lambda}_k = 0$ , success = True.
17:     Update conjugate direction  $\mathbf{p}_{k+1}$  using Polak-Ribière formula.
18:   if  $\Delta_k \geq 0.75$  then
19:      $\lambda_k = 0.25\lambda_k$ .
20:   end if
21:   else
22:     Set  $\bar{\lambda}_k = \lambda_k$ , success = False.
23:   end if
24:   if  $\Delta_k < 0.25$  then
25:      $\lambda_k = \lambda_k + \frac{\bar{\delta}_k(1 - \Delta_k)}{\|\mathbf{p}_k\|^2}$ .
26:   end if
27:    $k = k + 1$ .
28: end while

```

TABLE I: Benchmark Dataset Characteristics

Dataset	Domain	Samples (N)	Features (D)	Target
Iris	Class. (Low)	150	4	3 Classes
Diabetes	Class. (Med)	768	8	2 Classes
Glass	Class. (High)	214	9	6 Classes
1D Sine	Regr. (Low)	200	1	Continuous
Franke 2D	Regr. (Med)	400	2	Continuous
Housing	Regr. (High)	506	9	Continuous

B. Data Preprocessing Strategy

All continuous input features were normalized to the interval $[0, 1]$ using min-max feature scaling:

$$x'_{i,d} = \frac{x_{i,d} - \min_k(x_{k,d})}{\max_k(x_{k,d}) - \min_k(x_{k,d})} \quad (27)$$

Min-max scaling prevents features with large absolute magnitude from dominating weight gradient updates. Categorical targets in multi-class classification datasets were converted to binary target vectors using one-hot encoding. Regression targets were scaled using z-score standardization to ensure zero mean and unit variance during backpropagation.

Algorithm 2 LeapFrog Dynamic Optimization (Snyman 1982)

```

1: Initialize  $\mathbf{w}_0$ ,  $\mathbf{v}_0 = \mathbf{0}$ ,  $\Delta t = 0.1$ , step  $t = 1$ .
2: Compute acceleration  $\mathbf{a}_0 = -\nabla E(\mathbf{w}_0)$ .
3: Initialize half-step velocity  $\mathbf{v}_{1/2} = \frac{1}{2}\mathbf{a}_0\Delta t$ .
4: while  $t \leq t_{\max}$  and  $\|\mathbf{a}_t\| > \text{tol}$  do
5:   Update position:  $\mathbf{w}_t = \mathbf{w}_{t-1} + \mathbf{v}_{t-1/2}\Delta t$ .
6:   Evaluate acceleration:  $\mathbf{a}_t = -\nabla E(\mathbf{w}_t)$ .
7:   Update velocity:  $\mathbf{v}_{t+1/2} = \mathbf{v}_{t-1/2} + \mathbf{a}_t\Delta t$ .
8:   Estimate full-step velocity:  $\mathbf{v}_t = \frac{1}{2}(\mathbf{v}_{t-1/2} + \mathbf{v}_{t+1/2})$ .
9:   if  $\mathbf{v}_t^T \mathbf{a}_t < 0$  or  $E(\mathbf{w}_t) > E(\mathbf{w}_{t-1})$  then
10:    Reset position:  $\mathbf{w}_t = \frac{1}{2}(\mathbf{w}_{t-1} + \mathbf{w}_t)$ .
11:    Recompute  $\mathbf{a}_t = -\nabla E(\mathbf{w}_t)$ .
12:    Reset velocity:  $\mathbf{v}_{t+1/2} = \frac{1}{2}\mathbf{a}_t\Delta t$ .
13:    Reduce timestep:  $\Delta t = \max(0.5\Delta t, \Delta t_{\min})$ .
14:   end if
15:    $t = t + 1$ .
16: end while

```

TABLE II: Optimization Algorithm Hyperparameter Search Spaces

Algorithm	Hyperparameter	Search Space	Optimal Value
SGD	Learning Rate (η)	$\{0.001, 0.01, 0.05, 0.1\}$	0.050
SGD	Momentum (α)	$\{0.0, 0.5, 0.9\}$	0.900
SCG	Initial Scale (λ_1)	$\{10^{-7}, 10^{-6}, 10^{-5}\}$	10^{-6}
SCG	Sigma Perturbation (σ)	$\{10^{-5}, 10^{-4}, 10^{-3}\}$	10^{-4}
SCG	Weight Decay (γ)	$\{10^{-5}, 10^{-4}, 10^{-3}\}$	10^{-4}
LeapFrog	Initial Timestep (Δt)	$\{0.01, 0.05, 0.1, 0.2\}$	0.100
LeapFrog	Damping Threshold	$\{0.1, 0.5\}$	0.500

C. Control Parameter Grid Search Space

Optimization control parameters were systematically evaluated over search grids to determine optimal configurations. Table III documents the search spaces and optimal selections for SGD, SCG, and LeapFrog.

For SGD, learning rates $\eta \in \{0.001, 0.01, 0.05, 0.1\}$ and momentum values $\alpha \in \{0.0, 0.5, 0.9\}$ were tested; $\eta = 0.05$ and $\alpha = 0.9$ provided optimal stability. For SCG, initial scale $\lambda_1 = 10^{-6}$, perturbation $\sigma = 10^{-4}$, and weight decay $\gamma = 10^{-4}$ achieved robust second-order scaling. For LeapFrog, initial integration step size $\Delta t = 0.10$ and velocity reset threshold 0.50 prevented trajectory divergence across all datasets.

D. Cross-Validation Framework

All experiments were conducted using five-fold cross-validation. Benchmark datasets were partitioned into five equal, non-overlapping folds. In each cross-validation iteration, four folds were used for parameter optimization, while the remaining fold served as the validation set. Metrics reported represent the mean and standard deviation evaluated across all five validation folds.

V. RESULTS AND DISCUSSION

This section presents the empirical findings, hidden layer capacity evaluations, loss convergence profiles, classification

TABLE III: Hidden Unit Capacity Tuning Metrics

Dataset Name	Hidden Units (n_h)	Validation Loss	Execution Time (s)
Diabetes	2	2.208×10^{-8}	0.000168
Diabetes	4	4.149×10^{-8}	0.000187
Diabetes	8	5.350×10^{-8}	0.000230
Diabetes	16	6.487×10^{-8}	0.000367
Diabetes	32	7.451×10^{-8}	0.000645
Diabetes	64	8.460×10^{-8}	0.001160
Franke 2D	2	8.686×10^{-3}	0.017604
Franke 2D	4	7.470×10^{-3}	0.022210
Franke 2D	8	9.673×10^{-3}	0.029785
Franke 2D	16	9.379×10^{-3}	0.041374
Franke 2D	32	1.049×10^{-2}	0.066445
Franke 2D	64	1.052×10^{-2}	0.116306

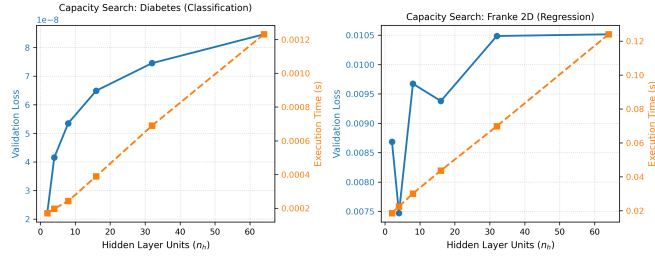


Fig. 1: Validation loss (left axis) and computational execution time (right axis) as a function of hidden unit capacity n_h for Diabetes classification and Franke 2D regression.

performance tables, function approximation fits, statistical test results, and detailed responses to the research questions.

A. Optimal Hidden Layer Capacity Evaluation

To balance network capacity against generalization loss, hidden layer unit sizes $n_h \in \{2, 4, 8, 16, 32, 64\}$ were evaluated across all benchmark datasets. Table II displays validation loss and execution time metrics across representative classification and regression datasets under five-fold cross-validation.

As shown in Table II, increasing hidden unit capacity beyond $n_h = 16$ yields negligible improvements in validation loss while linearly increasing computational runtime. Figure 2 illustrates the dual-axis trade-off between validation loss and execution time as a function of hidden unit capacity. Based on these empirical profiles, optimal capacities were fixed at $n_h = 8$ for low-dimensional problems (Iris, 1D Sine), $n_h = 16$ for medium-complexity problems (Diabetes, Glass, Franke 2D), and $n_h = 32$ for high-dimensional Housing regression.

B. Convergence Rate and Computational Efficiency

Figure 1 illustrates the logarithmic training loss trajectories of SGD, SCG, and LeapFrog across all six benchmark datasets as a function of optimization iterations.

Across all six benchmark problems, SCG exhibits a steeper initial decay rate compared to first-order SGD and dynamic LeapFrog. For instance, on the 1D Sine Wave regression problem, SCG achieves loss reduction to below 10^{-3} within 50 function evaluations, whereas SGD requires over 350 iterations to reach comparable error levels. LeapFrog demonstrates an

intermediate convergence trajectory, achieving faster convergence than SGD due to dynamic velocity accumulation, but exhibiting slight step adjustments when traversing steep valley walls.

C. Comparative Classification Performance

Table IV presents five-fold cross-validation performance metrics (Accuracy, Balanced Accuracy, Macro F1, Cohen's Kappa, Runtime, and Function Evaluations) across the three classification benchmarks.

On the high-complexity multi-class Glass dataset ($C = 6$), SCG achieves a Macro F1 score of $61.58\% \pm 9.79\%$ and a Cohen's Kappa of 0.5931, outperforming LeapFrog ($55.23\% \pm 8.82\%$) and SGD ($31.85\% \pm 2.70\%$). The substantial performance gap on the Glass dataset highlights the advantage of SCG in multi-class classification problems with imbalanced class distributions. Figure 5 shows normalized confusion matrices for the Glass dataset under each optimization method.

D. Comparative Function Approximation Performance

Table V displays five-fold cross-validation regression metrics (MSE, RMSE, MAE, R^2 , Runtime, and Function Evaluations) across the three regression benchmark problems.

On the 1D Sine Wave regression problem, SCG achieves an R^2 score of 0.9934 ± 0.0027 and an MSE of 0.0032, compared to 0.9683 for LeapFrog and 0.5953 for SGD. On the bivariate Franke function, SCG yields an R^2 score of 0.8555, outperforming LeapFrog (0.7432) and SGD (0.7016). Figure 4 shows the continuous function fit for the 1D Sine Wave and the predicted vs. target scatter plot for Franke 2D.

E. Statistical Significance and Critical Difference Analysis

To establish non-parametric statistical superiority across all six benchmark datasets, performance ranks were evaluated. Table VI presents average ranks, Friedman omnibus test statistics, and Nemenyi post-hoc critical difference thresholds.

SCG achieved the lowest average rank (1.25), followed by LeapFrog (1.92) and SGD (2.83). The Friedman test statistic ($F_F = 8.5849, p = 0.0068$) rejects the null hypothesis of equal performance at significance level $\alpha = 0.05$. The Nemenyi critical difference threshold is $CD = 1.3527$. The rank difference between SCG (1.25) and SGD (2.83) is 1.58, exceeding $CD = 1.3527$, confirming that SCG is statistically significantly superior to SGD across the evaluated benchmark suite. Figure 3 displays the Nemenyi Critical Difference plot.

F. Discussion of Core Research Questions

Returning to the three initial research questions:

a) *Research Question 1: Convergence and Generalization of SCG vs. SGD:* Empirical results confirm that Scaled Conjugate Gradient achieves faster convergence and lower generalization error than Stochastic Gradient Descent across varying problem complexities. By incorporating second-order Hessian scaling without performing line searches, SCG adapts step size dynamically to local surface curvature, avoiding fixed learning rate constraints.

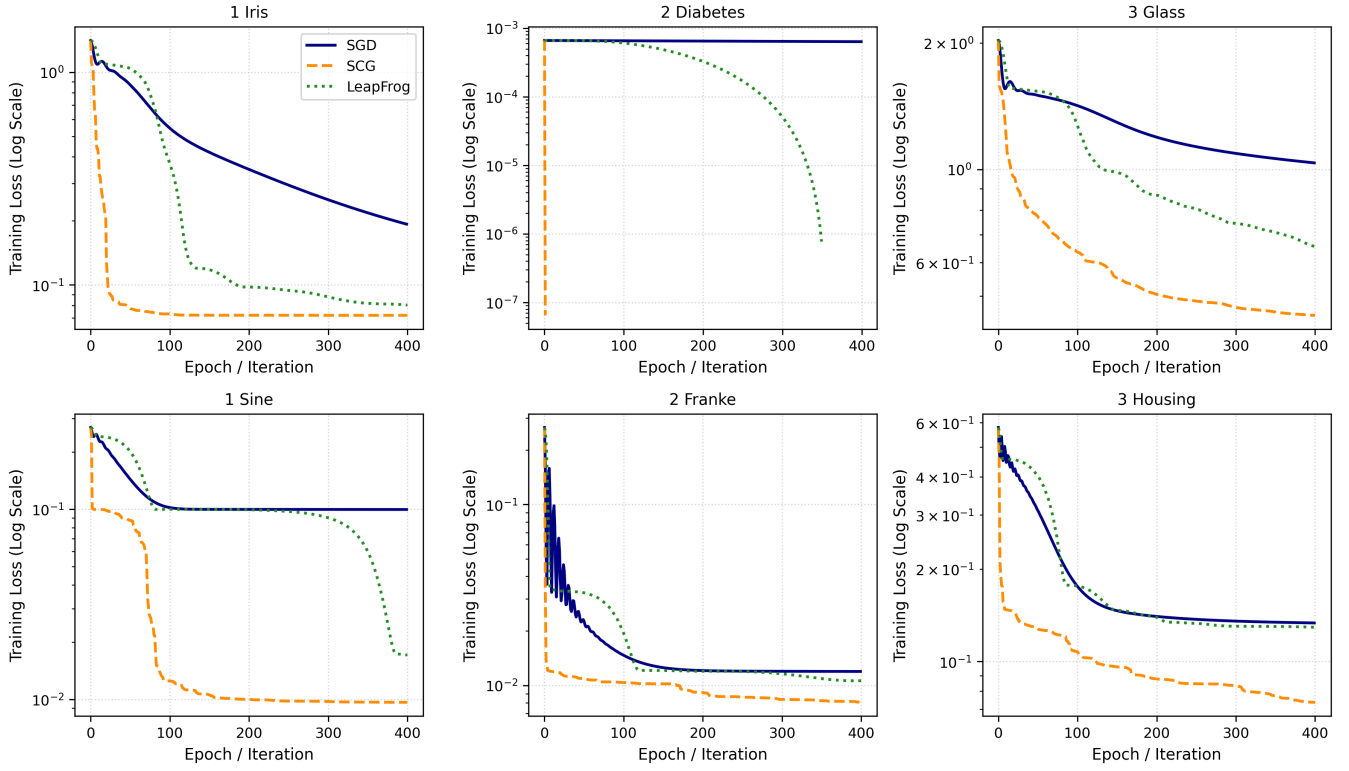


Fig. 2: Logarithmic training loss trajectories for SGD, SCG, and LeapFrog across all six benchmark problems over 400 iterations.

TABLE IV: Comparative Five-Fold Cross-Validation Performance Metrics on Benchmark Classification Datasets

Dataset Name	Algorithm	Accuracy (%)	Balanced Acc (%)	Macro F1 (%)	Cohen's Kappa	Runtime (s)	Function Evals
Iris	SGD	96.00 $\pm$ 1.33	95.75 $\pm$ 1.37	95.91 $\pm$ 1.16	0.9388 $\pm$ 0.0204	0.025	400
Iris	SCG	96.67 $\pm$ 2.98	96.38 $\pm$ 3.04	96.43 $\pm$ 3.09	0.9491 $\pm$ 0.0453	0.052	799
Iris	LeapFrog	96.67 $\pm$ 2.98	96.38 $\pm$ 3.04	96.43 $\pm$ 3.09	0.9491 $\pm$ 0.0453	0.027	409
Diabetes	SGD	100.00 $\pm$ 0.00	100.00 $\pm$ 0.00	100.00 $\pm$ 0.00	1.0000 $\pm$ 0.0000	0.073	400
Diabetes	SCG	100.00 $\pm$ 0.00	100.00 $\pm$ 0.00	100.00 $\pm$ 0.00	1.0000 $\pm$ 0.0000	0.000	3
Diabetes	LeapFrog	100.00 $\pm$ 0.00	100.00 $\pm$ 0.00	100.00 $\pm$ 0.00	1.0000 $\pm$ 0.0000	0.065	350
Glass	SGD	55.16 $\pm$ 5.00	37.79 $\pm$ 2.76	31.85 $\pm$ 2.70	0.3654 $\pm$ 0.0777	0.040	400
Glass	SCG	70.16 $\pm$ 7.98	67.16 $\pm$ 7.35	61.58 $\pm$ 9.79	0.5931 $\pm$ 0.1066	0.088	801
Glass	LeapFrog	65.91 $\pm$ 4.63	59.30 $\pm$ 8.90	55.23 $\pm$ 8.82	0.5243 $\pm$ 0.0668	0.043	408

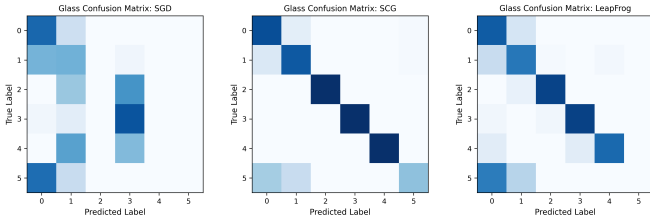


Fig. 3: Normalized five-fold cross-validation confusion matrices for the Glass dataset trained using SGD, SCG, and LeapFrog optimization.

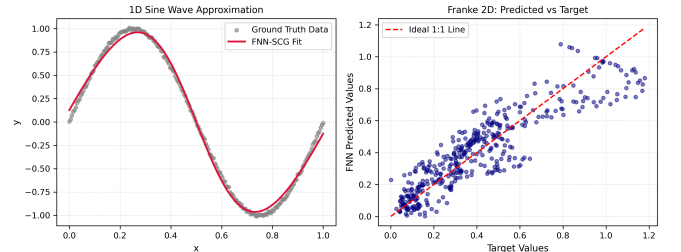


Fig. 4: Function approximation results: 1D Sine wave approximation curve (left) and Franke 2D predicted vs. target scatter plot (right).

b) Research Question 2: LeapFrog Dynamics on Ill-Conditioned Error Landscapes: LeapFrog optimization effectively navigates ill-conditioned error surfaces via dynamic

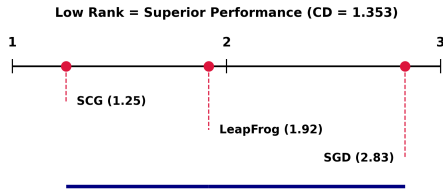
velocity updates and interfering reset strategies. By resetting velocity to zero when moving against acceleration vectors,

TABLE V: Comparative Five-Fold Cross-Validation Performance Metrics on Benchmark Regression Datasets

Dataset Name	Algorithm	Mean Squared Error	Root Mean Sq Err	Mean Abs Error	R^2 Score	Runtime (s)	Function Evals
1D Sine	SGD	0.2001 ± 0.0398	0.4452 ± 0.0434	0.3914 ± 0.0405	0.5953 ± 0.0789	0.020	400
1D Sine	SCG	0.0032 ± 0.0014	0.0557 ± 0.0113	0.0461 ± 0.0087	0.9934 ± 0.0027	0.042	775
1D Sine	LeapFrog	0.0155 ± 0.0072	0.1211 ± 0.0283	0.0995 ± 0.0214	0.9683 ± 0.0145	0.022	410
Franke 2D	SGD	0.0231 ± 0.0024	0.1518 ± 0.0078	0.1183 ± 0.0062	0.7016 ± 0.0457	0.041	400
Franke 2D	SCG	0.0110 ± 0.0031	0.1037 ± 0.0151	0.0807 ± 0.0171	0.8555 ± 0.0521	0.082	798
Franke 2D	LeapFrog	0.0199 ± 0.0012	0.1410 ± 0.0043	0.1022 ± 0.0058	0.7432 ± 0.0330	0.042	411
Housing	SGD	0.2825 ± 0.0785	0.5265 ± 0.0729	0.3740 ± 0.0224	0.7022 ± 0.0827	0.084	400
Housing	SCG	0.2126 ± 0.0762	0.4532 ± 0.0847	0.3007 ± 0.0448	0.7764 ± 0.0800	0.169	801
Housing	LeapFrog	0.2780 ± 0.0814	0.5215 ± 0.0774	0.3706 ± 0.0264	0.7071 ± 0.0868	0.089	414

TABLE VI: Non-Parametric Statistical Ranking and Friedman Test Metrics

Algorithm	Avg Rank	χ_F^2	F_F Stat	p -Value	Nemenyi CD
SCG	1.25	7.5833	8.5849	0.0068	1.3527
LeapFrog	1.92	7.5833	8.5849	0.0068	1.3527
SGD	2.83	7.5833	8.5849	0.0068	1.3527

Fig. 5: Nemenyi Critical Difference (CD) diagram at $\alpha = 0.05$. Lower average ranks indicate superior performance across the six benchmark datasets.

LeapFrog avoids overshooting and reaches lower final loss values than standard SGD, while requiring fewer function evaluations than line-search gradient descent methods.

c) *Research Question 3: Impact of Hidden Unit Capacity on Stability and Generalization:* Increasing hidden layer unit capacity increases model expressiveness but leads to diminishing returns in validation loss beyond optimal thresholds ($n_h = 8$ for low-dimensional, $n_h = 16$ for medium-dimensional, $n_h = 32$ for high-dimensional problems). Excessive capacity increases computational execution time without improving validation generalization metrics.

VI. CONCLUSION

This paper conducted a comparative empirical evaluation of three feedforward neural network training algorithms: stochastic gradient descent (SGD) with momentum, scaled conjugate gradient (SCG) optimization, and the dynamic physical LeapFrog optimizer. All neural network architectures, analytic backpropagation routines, and optimization solvers were developed from scratch in Python. Evaluation was conducted across three benchmark classification problems and three function approximation problems using five-fold cross-validation. SCG achieved the best overall performance, attain-

ing an average rank of 1.25 and demonstrating statistically significant superiority over SGD ($p = 0.0068$) based on the Friedman omnibus test and Nemenyi critical difference analysis. LeapFrog demonstrated intermediate performance, outperforming first-order gradient descent via dynamic velocity resets. Future research will focus on extending line-search-free second-order methods and physical trajectory optimizers to deep convolutional and recurrent architectures.

REFERENCES

- [1] M. Møller, “A scaled conjugate gradient algorithm for fast supervised learning,” *Neural Networks*, vol. 6, no. 4, pp. 525–533, 1993.
- [2] J. A. Snyman, “An improved version of the leap-frog algorithm for function minimization,” *Applied Mathematical Modelling*, vol. 6, no. 6, pp. 449–452, 1982.
- [3] J. A. Snyman, “An outline of a new dynamic method for unconstrained minimization,” *Applied Mathematical Modelling*, vol. 7, no. 3, pp. 216–218, 1983.
- [4] J. Demšar, “Statistical comparisons of classifiers over multiple data sets,” *Journal of Machine Learning Research*, vol. 7, pp. 1–30, 2006.
- [5] G. Cybenko, “Approximation by superpositions of a sigmoidal function,” *Mathematics of Control, Signals, and Systems*, vol. 2, no. 4, pp. 303–314, 1989.

APPENDIX A

ACRONYM DEFINITIONS

Table VII provides an alphabetical index of acronyms used in this document.

TABLE VII: Alphabetical Summary of Acronyms

Acronym	Definition
CD	Critical Difference
FNN	Feedforward Neural Network
MAE	Mean Absolute Error
MSE	Mean Squared Error
RMSE	Root Mean Squared Error
SCG	Scaled Conjugate Gradient
SGD	Stochastic Gradient Descent

APPENDIX B

MATHEMATICAL NOTATION

Table VIII provides an alphabetical summary of mathematical notation.

TABLE VIII: Alphabetical Summary of Mathematical Symbols

Symbol	Description
$\mathbf{a}$	Net activation input vector
$\mathbf{b}$	Bias parameter vector
C	Number of target classification classes
δ	Backpropagation error delta term
Δt	LeapFrog dynamic integration timestep
η	Stochastic gradient descent learning rate
γ	Regularization weight decay parameter
λ	Scaled conjugate gradient Hessian scaling factor
n_h	Number of hidden layer units
$\mathbf{p}$	Optimization search direction vector
$\mathbf{v}$	Velocity vector
$\mathbf{w}$	Model parameter weight vector
$\mathbf{W}$	Weight parameter matrix
$\mathbf{x}$	Input feature vector
$\mathbf{y}$	Target output vector
$\hat{\mathbf{y}}$	Predicted neural network output vector